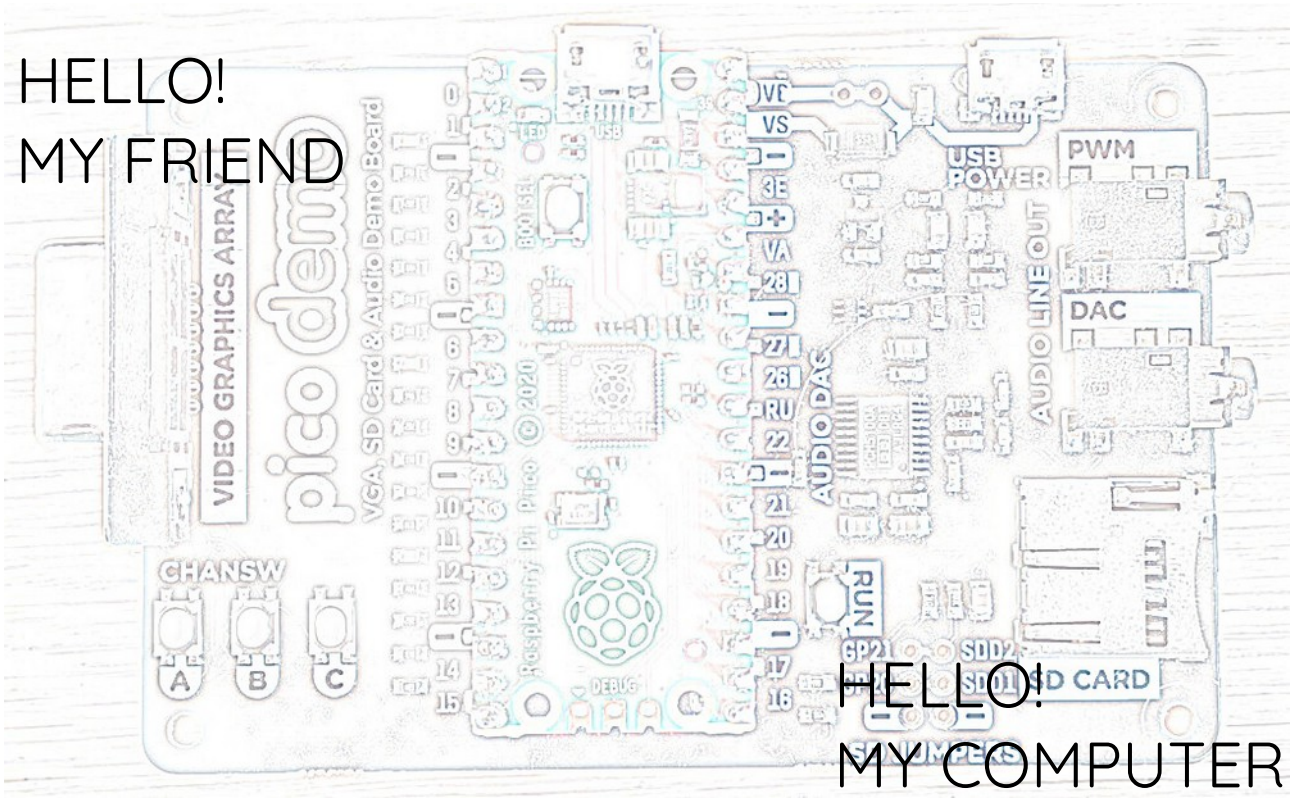
# Pico MZ

## User and Systems Manuals




Pico MZ Manual, Version 3.2.0, November 2026.
Copyright © 2024-2026 Tim Holyoake.

# Table of Contents


Introduction....4
Getting started....5
Hardware requirements – Pimoroni VGA Demo Base....5
Hardware requirements – RC2014 RP2040 VGA Terminal Card....7
Hardware requirements – RC2014 Pi Pico VGA Terminal Card....8
Finding software for the Pico MZ....9
Installing / re-installing the Pico MZ firmware....10
Pimoroni VGA demo base....10
RC2014 RP2040 VGA Terminal Card....10
RC2014 Pi Pico VGA Terminal Card....11
First boot – The MZ-80K is the default....12
The MZ-80A and MZ-700 can be selected at power on....12
Keyboard layouts....14
The MZ-80K keyboard layout....14
Mapping the MZ-80K keyboard to a UK USB keyboard....14
Black MZ-80K keys....14
Yellow MZ-80K keys....15
Blue MZ-80K keys....16
The MZ-80A keyboard layout....17
MZ-80A CTRL keys....17
The MZ-700 keyboard layout....19
MZ-700 CTRL keys....20
USB keyboard - Function keys....21
F1 to F3 – Cassette deck emulation....21
Shift F1 – Fast load for machine code programs (Type 0x01)....21
Shift F2 – Change SD card....21
F4 – Clear emulator status area....21
Shift F4 – UK/Japanese character graphics ROM (MZ-700 only)....21
F5 – Reverse video (MZ-80K only)....21
F6 – UK/Japanese character graphics ROM (MZ-80K only)....22
F5 - F9 – Sharp function keys (MZ-700 only)....22
F10 – Reset button (MZ-80A)....22
F10 – Reset button (MZ-700)....22
F11 and F12 – Experimental fast memory dump file....22
Unused function keys....23
Cassette tape emulation....24
MicroSD card layout....24
Loading files from a microSD card....24
Saving files to a microSD card....25
MicroSD card support....26
Troubleshooting....27
Systems manual....28
Compiling the Pico MZ Emulator....28
Pre-requisites for Raspberry Pi OS (Debian Bookworm and Trixie)....28
Pico MZ software architecture....30
Overview....30
The MZ-80K memory map....32




Copyright © 2024-2026 Tim Holyoake.

The MZ-80A memory map....33
The MZ-700 memory map....34
Memory dump files....35
Pico MZ memory dump file format as of November 2026 (v3.2.0)....35
Header....35
Monitor ROM or (MZ-80A / MZ-700) 4K banked RAM....36
Monitor workarea and user RAM....36
Video RAM....36
12K banked RAM (MZ-700 only)....36
Z80 state....36
8253 state....36
Release Highlights....37
October 2024: Release 1.0.0....37
November 2024: Release 1.1.0....37
January 2025: Release 1.2.0....37
January 2025: Release 1.2.3....37
January 2025: Release 1.2.4....37
February 2025: Release 2.0.0....37
November 2025: Release 3.0.0....37
September 2026: Release 3.1.0....37
November 2026: Release 3.2.0....37
Acknowledgements....38


Copyright © 2024-2026 Tim Holyoake.

# Introduction

The Sharp BASIC manual from 1979 introduces the user to the MZ-80K in this way.

> **Here’s a new friend for you**
>
> The MZ-80K is ready to enjoy conversation with you. Through conversation, it will help you solve difficult calculation problems or become a partner to play a game with. More than that, it has unknown potentialities to be opened up with you. This is just like a journey into unknown space. Together with your new friend, let’s make the journey now.

The Pico MZ aims to faithfully re-create this iconic computer as well as the later MZ-80A and MZ-700 models, so that your conversation can carry on more than four decades after the journey began.

But first, you will need to understand a little more about how to set up the Pico MZ, so that it can be your new friend.


Copyright © 2024-2026 Tim Holyoake.

# Getting started

## Hardware requirements – Pimoroni VGA Demo Base

The Pico MZ can use the Pimoroni VGA Demo Base with a Raspberry Pico or Pico 2 microcontroller to re-create the hardware of a Sharp MZ-80K, MZ-80A or MZ-700. To run the emulator, you will need:

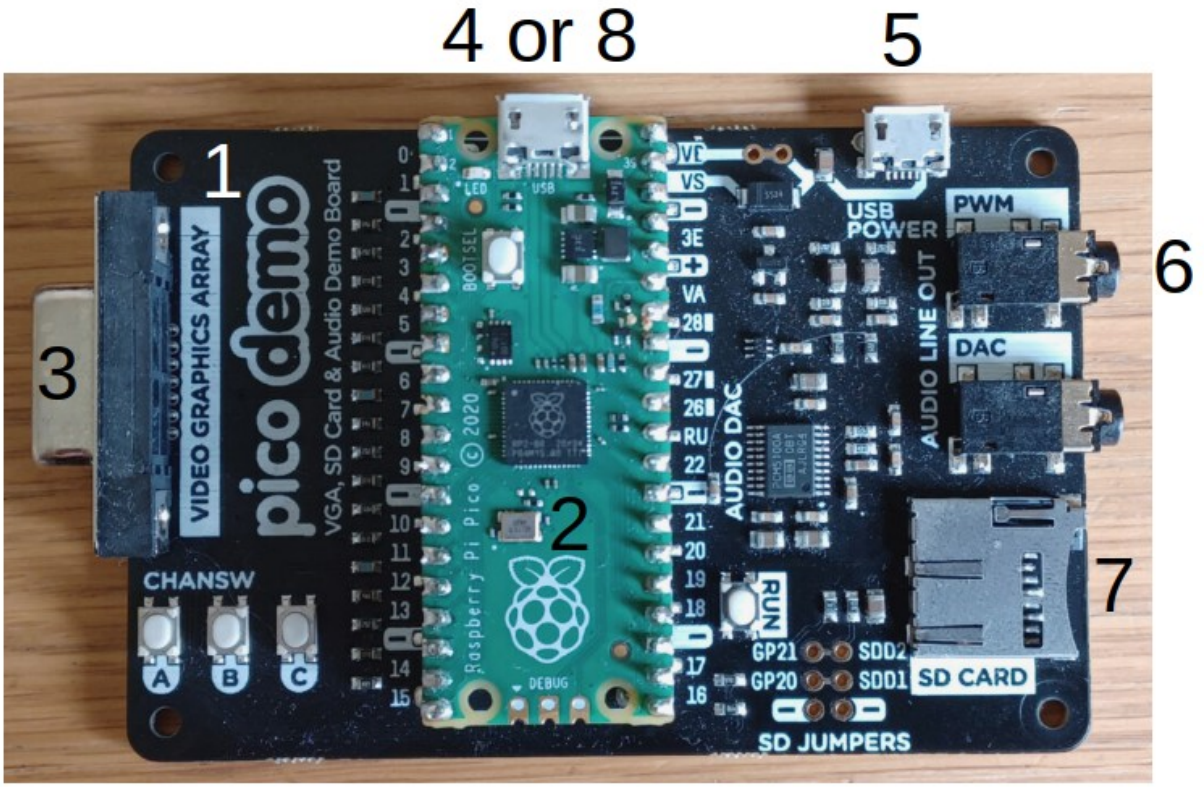
1. A Pimoroni VGA Demo Base.

2. A Raspberry Pico H or Pico2 H (headers soldered to a standard Pico or Pico 2).

3. A VGA cable to enable your VGA demo base to be plugged into a suitable monitor.

4. An OTG adaptor or cable to allow a USB keyboard to be plugged into the micro USB port on your Pico or Pico 2.

5. A power supply. This must be plugged into the micro USB port on your VGA demo base marked ‘USB POWER’. An official Raspberry Pi 5V, 12.5W Micro USB Power Supply is suitable.




6. A speaker or speakers, connected to a 3.5mm stereo jack, and plugged into the PWM socket on the VGA demo base. The DAC socket is not currently supported.


Copyright © 2024-2026 Tim Holyoake.

7. A FAT32 formatted microSD card[1], containing the Sharp MZ-80K, MZ-80A and/or MZ-700 software you wish to run. These should be ‘.mzf’, ‘.mzt’ or ‘.m12’ format files. Analogue .wav files are not currently supported by the emulator so must be converted before use.

8. A USB cable with a micro USB plug for the Pico, to enable the Pico MZ firmware to be installed and re-installed from a computer.

1 See the section on microSD card support for known working / not working microSD cards.


Copyright © 2024-2026 Tim Holyoake.

## Hardware requirements – RC2014 RP2040 VGA Terminal Card

The Pico MZ can use the RC2014 RP2040 VGA Terminal Card installed on a RC2014 backplane to re-create the hardware of a Sharp MZ-80K, Sharp MZ-80A or Sharp MZ-700. To run the emulator, you will need:

1. A RC2014 backplane.

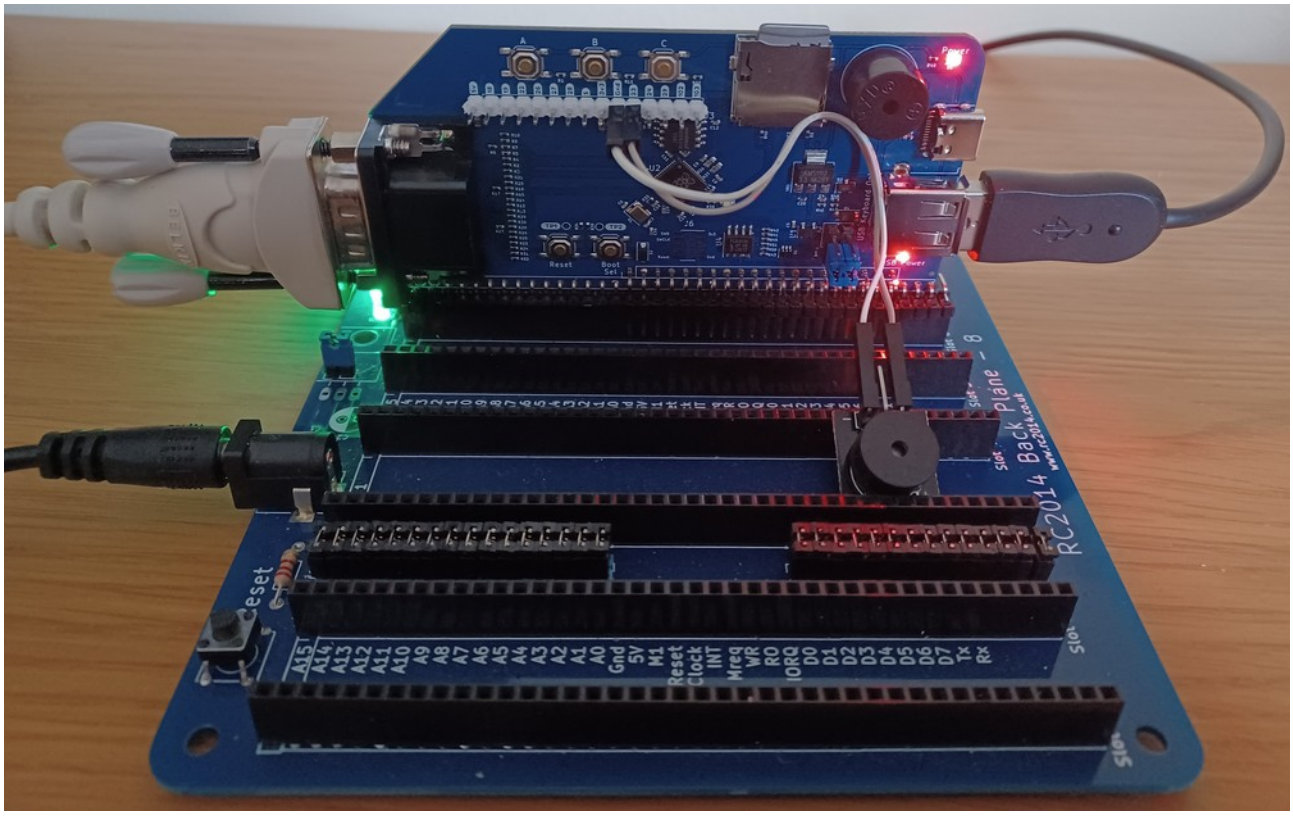
2. A RC2014 RP2040 VGA Terminal Card.

3. A VGA cable to enable your Terminal Card to be plugged into a suitable monitor.

4. A USB keyboard.

5. A 5v power supply for the RC2014 backplane.

6. For sound, a passive buzzer or speaker connected to GPIO23 and/or 24 on the expansion connector as the active buzzer on the card cannot be used.




7.A FAT32 formatted microSD card[2], containing the Sharp MZ software you wish to run. These should be ‘.mzf’, ‘.mzt’ or ‘.m12’ format files.

8. A USB cable with a USB-C plug for the RC2014 RP2040 VGA Terminal Card, to enable the Pico MZ firmware to be installed and re-installed from a computer. Note that the card cannot be powered using this port as the USB keyboard will not operate.

2 See the section on microSD card support for known working / not working microSD cards.


Copyright © 2024-2026 Tim Holyoake.

## Hardware requirements – RC2014 Pi Pico VGA Terminal Card

The Pico MZ can use the RC2014 Pi Pico VGA Terminal Card installed on a RC2014 backplane to re-create the hardware of a Sharp MZ-80K, MZ-80A or MZ-700. To run the emulator, you will need:

1. A RC2014 backplane.

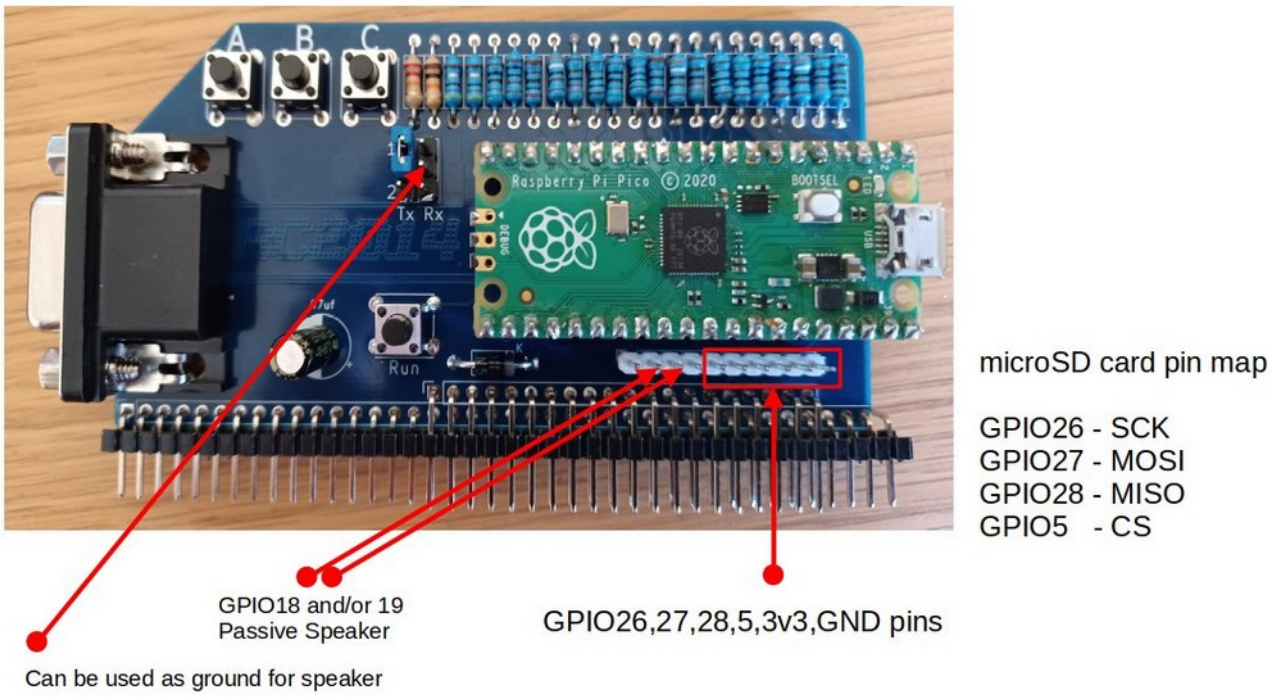
2. A RC2014 Pi Pico VGA Terminal Card plus a 3.3v capable microSD card breakout attached to the expansion connector.

3. A VGA cable to enable your Terminal Card to be plugged into a suitable monitor.

4. An OTG adaptor or cable to allow a USB keyboard to be plugged into the micro USB port on your Pico.

5. A 5v power supply for the RC2014 backplane.

6. For sound, a passive buzzer or speaker connected to GPIO18 and/or 19 on the expansion connector. Ground can be taken from the middle pin of the UART RX block, as the UART is not used by the emulator.




7.A FAT32 formatted microSD card[3], containing the Sharp MZ software you wish to run. These should be '.mzf', '.mzt' or '.m12' format files.

8. A USB cable with a micro USB plug for the RC2014 Pi Pico VGA Terminal Card, to enable the Pico MZ firmware to be installed and re-installed from a computer.

3 See the section on microSD card support for known working / not working microSD cards.


Copyright © 2024-2026 Tim Holyoake.

## Finding software for the Pico MZ

The Pico MZ is capable of running software designed for the Sharp MZ-80K, Sharp MZ-80A and Sharp MZ-700. The microSD card replaces the integrated cassette recorder found on the original machine, so digital copies of the software you wish to use are required.

Good sources of .mzf/.mzt/.m12 files for these machines include:

https://sharpmz.net/ (MZ-80K, MZ-80A and MZ-700)

https://mz-archive.co.uk/ (MZ-80K, MZ-80A and MZ-700)

https://mzpico.com/ (MZ-700 only)

https://github.com/psychotimmy/sharpmz-80k (MZ-80K only)

https://github.com/psychotimmy/Sharp-MZ-700 (MZ-700 only)

As a minimum, a language interpreter (such as Sharp's BASIC interpreter) or Z80 development environment (such as Avalon ZEN) should be written to the microSD card. The Pico MZ is of little use without such an interpreter or development environment.

A copy of the Sharp MZ-80K SP-5025 BASIC manual can be found at:

https://archive.org/details/sharp-basic-manual-mz-80-k

A copy of the Sharp MZ-80A Owners manual can be found at:

https://archive.org/details/Sharp-MZ-80A-owners-manual/

A copy the Sharp MZ-700 Owners manual can be found at:

https://archive.org/details/sharpmz700ownersmanual/


Copyright © 2024-2026 Tim Holyoake.

# Installing / re-installing the Pico MZ firmware

The most recent release of the Pico MZ firmware can be found at:

https://github.com/psychotimmy/picomz-80k/releases

## Pimoroni VGA demo base

To install, download one of:

**picomz-pimoroni.uf2** if you are using a Raspberry Pico

**pico2mz-pimoroni.uf2** if you are using a Raspberry Pico 2

Then:

1. Push and hold the BOOTSEL button while connecting your Pico/Pico 2 with a USB cable to a computer. Release the BOOTSEL button once your Pico appears as a Mass Storage Device called RPI-RP2 (RP2350 if you are using a Pico 2).
2. Copy the .uf2 file onto the RPI-RP2 (RP2350) volume. Your Pico/Pico 2 will reboot.
3. Disconnect the USB cable and plug in the OTG adaptor and USB keyboard to the Pico/Pico 2. Apply power to the USB POWER port.
4. You are now running the Pico MZ emulator.

## RC2014 RP2040 VGA Terminal Card

To install, download:

**picomz-rc2014.uf2**

Then:

1. Push and hold the BOOTSEL button while connecting your card with a USB cable to a computer. Release the BOOTSEL button once your card appears as a Mass Storage Device called RPI-RP2.
2. Copy the .uf2 file onto the RPI-RP2 volume. The card will reboot.
3. Disconnect the USB cable and plug in the USB keyboard. Apply power to the backplane.
4. You are now running the Pico MZ emulator.


Copyright © 2024-2026 Tim Holyoake.

## RC2014 Pi Pico VGA Terminal Card

To install, download:

**picomz-rc2014.uf2**

Then:

1. Push and hold the BOOTSEL button while connecting your Pico with a USB cable to a computer. Release the BOOTSEL button once your Pico appears as a Mass Storage Device called RPI-RP2.
2. Copy the .uf2 file onto the RPI-RP2 volume. Your Pico will reboot.
3. Disconnect the USB cable and plug in the OTG adaptor and USB keyboard to the Pico. Apply power to the USB POWER port.
4. You are now running the Pico MZ emulator.


Copyright © 2024-2026 Tim Holyoake.

## First boot – The MZ-80K is the default

If everything is working as expected, your VGA monitor should display:

```
** MONITOR SP-1002 **

*
```

Use the F1 key to cycle to a m/c code ‘tape’ to load (for example, the SP-5025 BASIC interpreter).

Type LOAD <return>

The ‘tape’ will take some time to load (although not quite as long as a real tape on a Sharp MZ-80K).

If you chose to load the SP-5025 BASIC interpreter, you should see the message:

```
* SHARP BASIC SP-5025

34680 BYTES

READY

```

### *The MZ-80A and MZ-700 can be selected at power on*

To boot the emulator as a MZ-80A, hold down the ‘A’ button on your carrier board for approximately a second (until the Num Lock light is lit) while power is applied. If everything is working as expected, your VGA monitor should display (in green):

```
** MONITOR SA-1510 **

*
```

Use the F1 key to cycle to a m/c code ‘tape’ to load (for example, the SA-5510 BASIC interpreter).

Type L<return>

The ‘tape’ will take some time to load (although not quite as long as a real tape on a Sharp MZ-80A).

If you chose to load the SA-5510 BASIC interpreter, you should see the message:

```
————————————————————————————————

   BASIC interpreter  SA-5510

   Copyright 1981 by Sharp Corp.

————————————————————————————————

 32492 Bytes

Ready

```

If you have a speaker connected to the board, a beep should also be heard when BASIC is loaded.


Copyright © 2024-2026 Tim Holyoake.

To boot the emulator as a MZ-700, hold down the ‘B’ or ‘C’ button on your carrier board for approximately a second (until the Num Lock and Caps Lock lights are lit) while power is applied. If everything is working as expected, your VGA monitor should display:

```
** MONITOR 1Z-013A **

*
```

Use the F1 key to cycle to a m/c code ‘tape’ to load (for example, the S-BASIC interpreter, 1Z-013B).

Type L<return>

The ‘tape’ will take some time to load (although not quite as long as a real tape on a Sharp MZ-700).

If you chose to load the S-BASIC interpreter, you should see the message:

```
   BASIC INTERPRETER  1Z-013B V1.0A
  COPYRIGHT (C) 1983 BY SHARP CORP.

 36439 BYTES FREE
Ready

```


Copyright © 2024-2026 Tim Holyoake.

# Keyboard layouts

## The MZ-80K keyboard layout

The Sharp MZ-80K keyboard has 78 keys, arranged in five rows. There are 14 keys on the bottom row, and 16 keys on each of the other four.

The rightmost 5 keys on each row are blue and allow access to 75 of the Sharp's character graphics.

The remainder of the bottom row are yellow, and implement the space bar, carriage return, shift keys, break key, cursor movement and editing functions.

The first 11 keys on each of the other four rows (with the exception of the 11th key on the second row from the bottom – SML/CAP) implement alphanumeric and punctuation characters. By default, the alpha characters are upper case.

The shift key enables the character on the top of the key to be used – for example, shift Q returns >, and shift S the heart graphic.

The SML/CAP key allows lower case characters to be used on the alpha keys or if a symbol is printed on the lower right hand side of a key, that symbol. If the SML/CAP key is selected, the led to the right of the MZ-80K keyboard (usually green when power is on) is turned red. When deselected, the led returns to green.




## Mapping the MZ-80K keyboard to a UK USB keyboard

### Black MZ-80K keys

With Caps Lock off, the lower case alpha keys on a USB keyboard are mapped to the upper case alpha keys on the MZ-80K keyboard. Numeric keys are mapped to the numeric keys as expected.

With Caps Lock on (or shift <character> used), most of the alphanumeric keys on a USB keyboard are mapped to the shifted MZ-80K keys (for example, shift <1> maps to !, shift <Q> maps to > and shift <S> maps to the heart symbol).


Copyright © 2024-2026 Tim Holyoake.

The exceptions are:

<shift> 3 – maps to £, rather than #

<shift> 6 – maps to π, rather than ^

<shift> 7 – maps to &, rather than ‘

<shift> 8 – maps to *, rather than (

<shift> 9 – maps to (, rather than )

<shift> 0 – maps to ), rather than π.

Where a punctuation symbol appears on the USB keyboard and there is a match on the Sharp MZ-80K keyboard, that key corresponds to the Sharp MZ-80K key. For example, ‘:’ maps to ‘:’ (<shift> O on the MZ-80K keyboard) and ‘@’ maps to ‘@’ (<shift> U>).

The SML/CAP key is mapped to the ‘~’ (tilde) key. When selected, the Pico’s green led is lit. This is equivalent to the Sharp MZ-80K’s power led turning red (from green). When the SML/CAP key is deselected, the Pico’s led is turned off.

## Yellow MZ-80K keys

The yellow keys are mapped as follows:

Left hand shift key – mapped to either USB shift key. Because shifted characters are taken care of automatically, use <Ctrl> L if a program is expecting **only** a left hand shift key as input.

Right hand shift key – mapped to either USB shift key. Because shifted characters are taken care of automatically, use <Ctrl> R if a program is expecting **only** a right hand shift key as input.

CLR – mapped to the End key

HOME – mapped to the Home key

INST – mapped to the Insert key

DEL – mapped to the Delete and Backspace keys. <Ctrl> H will also work.

SPACE – mapped to the spacebar

Cursor up, down, left, right – mapped to the cursor up, down, left, right keys respectively

BREAK – mapped to the PgDn key

SHIFT BREAK – mapped to the PgUp key

CR – mapped to the carriage return key and the numeric keypad’s Enter key. <Ctrl> M will also work.


Copyright © 2024-2026 Tim Holyoake.

## Blue MZ-80K keys

The blue graphics keys are mapped to Alt keys as shown in the table below.

| | | | | |
|---|---|---|---|---|
| Alt Q<br>Top left blue key | Alt W | Alt E | Alt R | Alt T<br>Top right blue key |
| Alt Y | Alt U | Alt I | Alt O | Alt P |
| Alt A | Alt S | Alt D | Alt F | Alt G |
| Alt H | Alt J | Alt K | Alt L | Alt M |
| Alt Z<br>Bottom left blue key | Alt X | Alt C | Alt V | Alt B<br>Bottom right blue key |

In common with the black keys, <shift><Alt><key> selects the symbol on the top of the MZ-80K key. If SML/CAP is active (steady green led lit on Pico), the symbol on the bottom right of the MZ-80K key is selected instead.


Copyright © 2024-2026 Tim Holyoake.

# The MZ-80A keyboard layout

The Sharp MZ-80A keyboard has 73 keys consisting of a main keyboard of five rows and a four row numeric keypad. This makes it close to, but not exactly the same as, modern UK USB keyboard layouts.

The MZ-80A keyboard has two operating modes:

1. **Standard mode.** This is selected at power on. The keyboard defaults to upper case – lower case letters can be selected by using a ‘shift’ key. The numeric keypad on a UK USB keyboard is in NUM LOCK mode by default.
2. **Graphics mode.** This is activated by pressing the ‘Esc’ key on a UK USB keyboard. The cursor changes shape and the keyboard allows many of the Sharp graphics characters to be accessed, either directly or through using a ‘shift’ key. To return to standard mode, press ‘Esc’ again.

If you are using a UK USB keyboard with the emulator, the standard mode mappings reflect the printed keyboard legends with the following exceptions:

‘Esc’ – mapped the Sharp GRPH key – toggles between standard mode and graphics mode.

<shift> 3 – mapped to the Sharp # key, rather than £.

End – mapped to the Sharp CLR key.

PgUp and PgDn – both mapped to the Sharp BREAK key.

Numeric keypad * - mapped to the Sharp 00 numeric keypad key.

Numeric keypad / - mapped to the up arrow (↑) character.

Numeric keypad <shift> / - mapped to the left arrow ( ← ) character.

## MZ-80A CTRL keys

If you are using a physical USB keyboard connected to the emulator, then the Ctrl key acts as the Sharp CTRL key.

The CTRL keys implemented are:

CTRL A – Shift lock (toggle)

CTRL D – Rolls up the display (MZ-80A display mode only)

CTRL E – Rolls down the display (MZ-80A display mode only)

CTRL Q – Cursor down

CTRL R – Cursor up

CTRL S – Cursor right

CTRL T – Cursor left


Copyright © 2024-2026 Tim Holyoake.

CTRL U – Cursor home

CTRL V – Cursor home and clear screen

CTRL Z – Prints the right arrow (→) character to the screen

CTRL @ (mapped to Ctrl ‘ or Alt ‘) - Reverse video (toggle)

CTRL [ - Set the VRAM configuration to MZ-80K mode (Top left screen address fixed at 0xD000)

CTRL ] - Set the VRAM configuration to MZ-80A mode (Top left screen address floats)


Copyright © 2024-2026 Tim Holyoake.

## The MZ-700 keyboard layout




The Sharp MZ-700 keyboard has 69 keys consisting of a main keyboard of five rows, five blue function keys plus six keys on the right hand side that control cursor movement. This makes it close to, but not exactly the same as, modern UK USB keyboard layouts.

The MZ-700 keyboard has two operating modes:

1. **Standard mode.** This is selected at power on. The keyboard defaults to upper case. Lower case letters can be selected by using any ‘shift’ key. In addition, the USB CAPS LOCK key toggles the keyboard between upper and lower case characters[4] (equivalent to typing SHIFT ALPHA on the MZ-700 keyboard) if this is supported by the program being used (e.g. the standard monitor ROM does not, but S-BASIC does). The state of this toggle is confirmed by the USB keyboard CAPS LOCK light.

   The numeric keypad on a UK USB keyboard is in NUM LOCK mode by default. The mode is confirmed by the USB keyboard NUM LOCK light.

2. **Graphics mode.** This is activated by pressing the ‘TAB’ key on a UK USB keyboard. The cursor changes shape and the keyboard allows many of the Sharp graphics characters to be accessed, either directly or through using a ‘shift’ key. To return to standard mode, press the CAPS LOCK key.

Standard mode mappings reflect the printed USB keyboard legends with the following exceptions:

‘TAB’ – mapped to the Sharp GRAPH key – selects graphics mode.

‘CAPS LOCK’ – when in standard mode, toggles between upper and lower case characters. When in graphics mode, the keyboard is returned to standard mode. Upper case is selected.

End – mapped to the Sharp CLR key.

PgDn – mapped to the Sharp BREAK key

4 Or Japanese characters, if the Japanese character set has been selected using the Shift F4 CGROM toggle.


Copyright © 2024-2026 Tim Holyoake.

PgUp – mapped to the Sharp SHIFT BREAK key combination

^ (shift 6) – mapped to the up arrow (↑) character.

_ (shift -) - mapped to the pi (π) character.

¬ (shift `) - mapped to the down arrow (↓) character.

Numeric keypad / - mapped to the up arrow (↑) character.

Numeric keypad <shift> / - mapped to the left arrow (←) character.

Numeric keypad <shift> * - mapped to the right arrow (→) character.

## MZ-700 CTRL keys

The USB Ctrl key maps to the Sharp CTRL key.

The CTRL keys implemented are:

CTRL E – Set keyboard to lower case

CTRL F – Set keyboard to upper case

CTRL M - <CR>

CTRL P - <DEL>

CTRL Q – Cursor down

CTRL R – Cursor up

CTRL S – Cursor right

CTRL T – Cursor left

CTRL U – Cursor home

CTRL V – Cursor home and clear screen

CTRL W - <GRAPH>

CTRL X - <INST>

CTRL Y - <ALPHA>

CTRL F10 = CTRL <reset button> - MZ-700 cold start.


Copyright © 2024-2026 Tim Holyoake.

# USB keyboard - Function keys

The USB function keys allow the following tasks to be accomplished on the Pico MZ.

## F1 to F3 – Cassette deck emulation

F1 – Step ‘forwards’ through files on the microSD card ‘tape’.

F2 – Step ‘backwards’ through files on the microSD card ‘tape’.

When F1 or F2 are pressed, the next tape file that a LOAD command will use is displayed in the emulator status area.

F3 – Reset the tape counter in the emulator status area to 000.

Note: F3 does not affect the next tape file that a LOAD (or fast load) command will use.

## Shift F1 – Fast load for machine code programs (Type 0x01)

Shift F1 – Loads the machine code program displayed in the emulator status area directly into memory and executes it from the address specified in the tape header. If a file other than a machine code executable (type 0x01) is selected, it is not loaded and an error message is output to the emulator status area.

## Shift F2 – Change SD card

Shift F2 – Allows an SD card to be hot-swapped. Remove the current SD card and use Shift F2 to make a new SD card visible to the emulator.

## F4 – Clear emulator status area

F4 – Clears the emulator status area when pressed.

Note: F4 does not affect the next tape file that a LOAD (or fast load) command will use.

## Shift F4 – UK/Japanese character graphics ROM (MZ-700 only)

Shift F4 – Toggles between the UK (default) and Japanese character graphics ROM.

## F5 – Reverse video (MZ-80K only)

F5 – Toggles between normal and reverse video.

While this was not possible on a standard MZ-80K, a Sharp Users’ Club article presented the hardware modifications necessary to implement it.

On the MZ-80A reverse video is implemented as standard by selecting CTRL @ (ctrl ‘ on a UK USB keyboard).

Reverse video is not available for the MZ-700.


Copyright © 2024-2026 Tim Holyoake.

## F6 – UK/Japanese character graphics ROM (MZ-80K only)

F6 – Toggles between the UK (default) and Japanese character graphics ROM.

## F5 - F9 – Sharp function keys (MZ-700 only)

F5, F6, F7, F8 and F9 are mapped to the blue Sharp MZ-700 function keys F1, F2, F3, F4 and F5 respectively. The shift key works as expected with these keys. While the Sharp MZ-700 1Z-013A ROM monitor ignores these keys, 1Z-013B BASIC (S-BASIC) maps these keys as follows:

| | |
|---|---|
| USB F5 (F1) – RUN <CR> | Shift USB F5 (Shift F1) - CHR$( |
| USB F6 (F2) – LIST | Shift USB F6 (Shift F2) – DEF KEY( |
| USB F7 (F3) – AUTO | Shift USB F7 (Shift F3) - CONT |
| USB F8 (F4) – RENUM | Shift USB F8 (Shift F4) - SAVE |
| USB F9 (F5) – COLOR | Shift USB F9 (Shift F5) - LOAD |

## F10 – Reset button (MZ-80A)

To reset the emulator back to the monitor, press the F10 key. This emulates the action of the reset button on the MZ-80A.

## F10 – Reset button (MZ-700)

To warm start the emulator (the program counter is reset to 0x0000 but the contents of memory are not affected) press the F10 key. To cold start the emulator (the equivalent of a power off / power on) hold the Ctrl key down while pressing the F10 key. This emulates the action of the (Ctrl and) reset button on the MZ-700.

## F11 and F12 – Experimental fast memory dump file

F11 – Read a memory dump file (/memorydumps/mzdump<x><y>.mzd) from the microSD card.

This restores the state of all memory, the z80 CPU and 8253 PIT to the point at which the memory dump was created.

For Pico 1 (rp2040) based boards, <x> is always 1.

For Pico 2 (rp2350) based boards, <x> is always 2.

For MZ-80K memory dumps, <y> is always K.

For MZ-80A memory dumps, <y> is always A.

For MZ-700 memory dumps, <y> is always 7.

For example, a MZ-80A memory dump file created on a Pico 2 will be located in the microSD card's /memorydump directory as mzdump2A.mzd. A MZ-700 memory dump file created on a Pico 1 will be located in the same directory as mzdump17.mzd.


Copyright © 2024-2026 Tim Holyoake.

F12 – Store the contents of all memory and the z80 state to a memory dump file (/memorydumps/mzdump<x><y>.mzd).

Note that the previous contents of this file are always silently overwritten by F12. Only a single memory dump file of the same type per microSD card is currently supported.

Memory dump files created under emulator version 3.2.0 and later are not compatible with those created under earlier versions and vice-versa.

Memory dump files are not compatible across MZ emulator and Pico platform types.

## Unused function keys

F7, F8, F9 and F10 are not used by the Pico MZ in MZ-80K mode.

F5, F6, F7, F8 and F9 are not used by the Pico MZ in MZ-80A mode.


Copyright © 2024-2026 Tim Holyoake.

# Cassette tape emulation

## MicroSD card layout

By default, all Sharp MZ program and data files (.mzf/.mzt/.m12) are expected to be in the root directory of your microSD card. Any new programs or data files written to the card by the emulator will also be stored in the root directory.

For convenience, it is also possible to separate files for the MZ-80K, MZ-80A and MZ-700 on a single card. To do this, create any or all of the sub-directories /mz80k, /mz80a and /mz700 on your microSD card and store your program and data files in the appropriate sub-directory.

If the emulator is running as a MZ-80K and the /mz80k directory is present, then ONLY this sub-directory will be examined for program and data files (i.e. the card's root directory will NOT be read). Any new files written by the emulator to the microSD card will be placed into this directory.

The same is true if you run the emulator as a MZ-80A and the /mz80a sub-directory is present, and similarly for MZ-700 emulation.

Experimental memory dump files are ALWAYS stored in and retrieved from the /memorydumps directory on the microSD card. If this sub-directory does not exist the first time a memory dump is created, then the /memorydumps sub-directory is created automatically.

Memory dump files from different emulators / Pico types are identified using a naming convention documented later this manual, but this naming convention is completely transparent to the user.

## Loading files from a microSD card

The microSD card acts in much the same way as a cassette tape works on a real Sharp MZ.

Use the F1 key to position the tape read head at the start of a new file. This is the equivalent of using the fast forward key and tape counter on a real machine. Repeatedly pressing F1 will cycle forwards through all of the files on the microSD card before stopping at the last file.

Use the F2 key to position the tape read head at the tape file before the current one. Repeatedly pressing F2 will cycle backwards through the files on the microSD card until the first file is reached.

F3 resets the tape counter. It does not affect the currently selected tape file – use F1 or F2 to change the tape file selected.

The bottom five lines of the Pico MZ's display are used to display 'tape' status. The name of the next file to be loaded is displayed (note that this is not necessarily the same name that the file has on the microSD card), along with the file type (one of m/c code, BASIC etc. data or unknown).

Files of type m/c code must be read directly from the monitor.

Files of type Sharp BASIC etc. must be read from the appropriate interpreter or development environment.


Copyright © 2024-2026 Tim Holyoake.

Files of type data are for use by the originating program. For example, the game “The Valley” allows your character to be saved and loaded from tape. These are stored in files of type data.

Use the LOAD command when in the monitor or BASIC (or the equivalent if you are using another interpreter or development environment) to transfer this file into the Pico MZ’s memory.

Unlike on a real Sharp MZ, using the LOAD command simulates you pressing the PLAY button on the cassette deck automatically, and stops once the end of the file is reached.

Loading files from the microSD card takes a little time as it is emulating a real tape. However, tapes are read slightly more quickly than on a real machine.

The F4 key will clear the ‘tape’ status display until the next time F1, F2, F3 or F9 is pressed.

## Saving files to a microSD card

Use the SAVE command when in BASIC (or the equivalent if you are using another interpreter or development environment) to write the contents of the Pico MZ’s memory to the microSD card.

The name of the file saved to the microSD card is not necessarily the same as the name given to the SAVE command. This is because the permitted characters in a FAT32 file name are not equivalent to the ones permitted in Sharp MZ file names (and vice-versa).

Note that if a file on your microSD card already exists it will be overwritten without warning.


Copyright © 2024-2026 Tim Holyoake.

# MicroSD card support

The following microSD cards and formats are known to work in the emulator.

| **microSD card make / type** | **microSD card format and partition sizes** |
|---|---|
| SanDisk Ultra 8GB microSDHC, Class 10, UHS 1 | FAT32, partition sizes up to and including the whole card |
| Transcend 16GB microSDHC, Class 10, UHS 1 | FAT32, partition sizes up to and including the whole card |
| Kingston 32GB microSDHC, Class 10, UHS 1 | FAT32, 2GB partition size |

The following microSD cards and formats are known **not** to work in the emulator.

| **microSD card make / type** | **microSD card format and partition sizes** |
|---|---|
| SanDisk Ultra 32GB microSDHC Class 10, A1, UHS 1 | FAT32, all partition sizes |


Copyright © 2024-2026 Tim Holyoake.

# Troubleshooting

| Symptom | Likely cause | Remedy |
|---|---|---|
| Pimoroni VGA Demo Base & RC2014 Pi Pico VGA card: Fast flashing green led (200ms) on the Pico or Pico 2; no output seen on the VGA display. | A USB keyboard is not active. | Check connections and try again by pressing the RUN button. |
| Pimoroni VGA Demo Base & RC2014 Pi Pico VGA card: Slow flashing green led (1s) on the Pico or Pico 2; no output seen on the VGA display. | There is no microSD card present, or the microSD card cannot be read. | Review the manual section that discusses SD card support. |
| RC2014 RP2040 Terminal Card: Fast flashing white led (200ms); red USB power led not lit; no output seen on the VGA display. | A USB keyboard is not active. | Check connections and try again by pressing the RESET button on the card. |
| RC2014 RP2040 Terminal Card: Slow flashing white led (1-2s) on; “Mounting SD card … failed. Try again.” message on screen. | There is no microSD card present, or the microSD card cannot be read. | Review the manual section that discusses SD card support. |
| ‘tapes’ fail to load or save correctly. | The cassette tape deck emulation is out of synchronisation. | Press the ‘BREAK’ key (PgDn) or the shifted ‘BREAK’ key (PgUp) and try again.<br><br>If this fails, restart the emulator by pressing the RUN or RESET button, depending on your hardware. |
| ‘tapes’ fail to save correctly and the cassette tape deck emulation is not out of synchronisation. | The microSD card (or partition being used on the card) is full. | Remove the microSD card from the emulator and delete unwanted files. Reinsert the card and try again. |


Copyright © 2024-2026 Tim Holyoake.

# Systems manual

## Compiling the Pico MZ Emulator

### Pre-requisites for Raspberry Pi OS (Debian Bookworm and Trixie)

CMake (version 3.13 or later) and a gcc cross compiler.

```
sudo apt install cmake

sudo apt install gcc-arm-none-eabi libnewlib-arm-none-eabi build-essential
```

The Pico MZ relies on the Raspberry Pico SDK. At the current time version 2.2.0 is required as there are issues in later versions that can hang the emulator until a key is pressed. This, and the Pico Extras repository, must be available on your computer if you wish to compile the emulator.

Assuming that you are already in the subdirectory in which you wish to install the Pico SDK, Pico Extras and Pico MZ repositories, issue the commands:

```
git clone --recursive https://github.com/raspberrypi/pico-sdk.git -b 2.2.0
git clone https://github.com/raspberrypi/pico-extras.git -b master
```

Then clone **either** the current release of the Pico MZ repository:

```
git clone https://github.com/psychotimmy/picomz-80k.git -b <current version>
```

**or** the latest stable version:

```
git clone https://github.com/psychotimmy/picomz-80k.git -b main
```

Ensure that the Pico SDK and Pico Extras subdirectories have been exported.

For example, if these libraries have been installed under /home/pi, use:

```
export PICO_SDK_PATH=/home/pi/pico-sdk
export PICO_EXTRAS_PATH=/home/pi/pico-extras
```

For a Pico build, issue the commands:

```
cd picomz-80k
mkdir build2040
cd build2040
cmake -DPICO_PLATFORM=rp2040 ..
make
```

For a Pico 2 build, issue the commands:

```
cd picomz-80k
mkdir build2350
cd build2350
cmake -DPICO_PLATFORM=rp2350 ..
make
```


Copyright © 2024-2026 Tim Holyoake.

There should now be two (Pico) or one (Pico 2) .uf2 files in your build directory for the emulator that can be installed on the appropriate hardware.


Copyright © 2024-2026 Tim Holyoake.

# Pico MZ software architecture

## Overview

<table>
<tr><td rowspan="9">sharpmz.h<br><br>Common header file for the emulator</td><td>sharpmz.c or sharpmz700.c<br><br>Main entry point for the emulator.</td></tr>
<tr><td>sharpcorp.c<br><br>The decoded SP-1002 and SA-1510 monitors and computer graphics ROMs (MZ-80K UK, MZ-80K2E Japanese and MZ-80A UK versions).<br><br>or sharpcorp 700.c<br><br>The decoded 1Z-013A monitor and UK/European computer graphics ROM for the MZ-700.</td></tr>
<tr><td>8255.c<br><br>A simplified Intel 8255 emulator, specifically for use in this emulator.</td></tr>
<tr><td>8253.c<br><br>A simplified Intel 8253 emulator, specifically for use in this emulator. The mechanism for producing sounds from the Pico’s PWM is also included in this source file.</td></tr>
<tr><td>keyboard.c + tusb_config.h<br><br>Emulates the Sharp MZ-80K and MZ-80A keyboards on a UK USB keyboard.<br><br>or keyboard700.c + tusb_config.h<br><br>Emulates the Sharp MZ-700 keyboard on a UK USB keyboard.</td></tr>
<tr><td>cassette.c<br><br>Emulates reading and writing Sharp MZ-80K and MZ-80A tapes (.mzf /.mzt / .m12 format, including the more common BSD files) using the VGA board’s microSD card.</td></tr>
<tr><td>vgadisplay.c<br><br>A monochrome VGA representation of the Sharp MZ-80K and MZ-80A’s display<br><br>or vgadisplay700.c<br><br>An eight colour VGA representation of the Sharp MZ-700’s display<br><br>Emulator status information is provided using the lower 40 scanlines in both versions.</td></tr>
<tr><td>miscfuncs.c<br><br>Miscellaneous functions used by the emulator.</td></tr>
<tr><td>pca9536.c<br><br>Driver software for the PCA9536D chip found on the RC2014 RP2040 VGA Terminal card. Only used by this board. Original source was the RC2014 picoterm firmware, https://github.com/RC2014Z80/picoterm</td></tr>
<tr><td colspan="2">Third party libraries (see next page)</td></tr>
</table>


Copyright © 2024-2026 Tim Holyoake.

**Third party libraries**

**zazu80** – a z80 instruction set emulator.
Forked from https://github.com/superzazu/z80

**fatfs** – a file system for the Raspberry Pico microSD card.
Version 0.16 w/ patch 2 forked from https://elm-chan.org/fsw/ff/

**sdcard** – low level routines for the fatfs library.
Forked from https://github.com/elehobica/pico_fatfs with changes made to support the pinout used by the Pimoroni VGA demo base and RC2014 VGA sd card.

**Raspberry Pi Pico libraries**

Pico SDK – Version 2.2.0 master branch, including TinyUSB. Pico SDK 2.3.0 and 2.3.1 are not currently supported.
Pico Extras – master branch.


Copyright © 2024-2026 Tim Holyoake.

## The MZ-80K memory map

| MZ-80K Addresses | | Pico MZ-80K Emulator |
|---|---|---|
| **0xF000 – 0xFFFF (61440 – 65535)**<br><br>**FD ROM uses first 1024 bytes of this space when installed** | FD ROM and unused addresses | Not implemented |
| **0xE000 – 0xEFFF (57344 – 61439)**<br><br>**Only addresses 0xE000 – 0xE008 used** | Devices<br><br>(8255, 8253, Sound) | Implemented by 8255.c and 8253.c |
| **0xD000 – 0xDFFF (53248 – 57343)**<br><br>**Only addresses 0xD000 – 0xD3FF populated with RAM** | 1kVideo RAM | Stored in mzvram[] |
| **0x1200 – 0xCFFF (4608 – 53247)** | User RAM | Stored in element 512 onwards of mzuserram[] |
| **0x1000 – 0x11FF (4096 - 4607)** | Monitor stack and workarea | Stored in elements 0 - 511 of mzuserram[] |
| **0x0000 – 0x0FFF (0 - 4095)** | Monitor ROM | Stored in mzmonitor80k[] |


Copyright © 2024-2026 Tim Holyoake.

## The MZ-80A memory map

| MZ-80A Addresses | | Pico MZ-80A Emulator |
|---|---|---|
| **0xF000 – 0xFFFF (61440 – 65535)**<br><br>**FD ROM uses first 2048 bytes of this space when installed** | FD ROM and unused addresses | Not implemented |
| **0xE000 – 0xEFFF (57344 – 61439)**<br><br>**Addresses 0xE000 – 0xE008 as for MZ-80K.** | Addresses 0xE800 – 0xEFFF reserved for user ROM option (not implemented)<br><br>Devices<br><br>(8255, 8253, Sound, Reverse Video, ‘A’ mode VRAM scrolling) | Implemented by 8255.c and 8253.c |
| **0xD000 – 0xDFFF (53248 – 57343)**<br><br>**Only addresses 0xD000 – 0xD7FF populated with RAM** | 2k Video RAM | Stored in mzvram[] |
| **0x1200 – 0xCFFF (4608 – 53247)** | User RAM<br><br>Addresses 0xC000 – 0xCFFF can be used to swap the monitor ROM – in which case, 0x0000-0x0FFF is remapped as RAM from this address space. | Stored in element 512 onwards of mzuserram[] |
| **0x1000 – 0x11FF (4096 - 4607)** | Monitor stack and workarea | Stored in elements 0 - 511 of mzuserram[] |
| **0x0000 – 0x0FFF (0 - 4095)** | Monitor ROM | Stored in mzmonitor80a[] |


Copyright © 2024-2026 Tim Holyoake.

## The MZ-700 memory map

| MZ-700 Addresses | | Pico MZ-700 Emulator |
|---|---|---|
| **0xD000 - 0xFFFF** | 12k Banked RAM – replaces the video RAM, devices and FD ROM space if switched in. | Implemented in picomz700.c. Stored in mzbank12[]. |
| **0xF000 - 0xFFFF** | FD ROMs etc. | Not implemented. |
| **Addresses 0xE000 – 0xE008 as for MZ-80K.** | Devices (8255, 8253, Sound) | Implemented by 8255.c and 8253.c |
| **0xD000 – 0xDFFF (53248 – 57343)** | 4k Video RAM | Stored in mzvram700[] |
| **0x1200 – 0xCFFF (4608 – 53247)** | User RAM | Stored in element 512 onwards of mzuserram[] |
| **0x1000 – 0x11FF (4096 - 4607)** | Monitor stack and workarea | Stored in elements 0 - 511 of mzuserram[] |
| **0x0000 – 0x0FFF (0 - 4095)** | 4k Banked RAM | Stored in mzbank4[]. |
| **0x0000 – 0x0FFF (0 - 4095)** | 4k Monitor ROM | Stored in mzmonitor700[] |

The areas of memory shaded in light grey are active at MZ-700 power on and after pressing the reset key (F10) at the same time as the Ctrl key.

The 4k and 12k banked RAM areas can be swapped into the 64K address space under program control. For example, the S-BASIC interpreter 1Z-013B has its own monitor in RAM, so the monitor ROM is swapped out for the 4k banked RAM when loaded.


Copyright © 2024-2026 Tim Holyoake.

# Memory dump files

The Pico MZ, through the F12 function key, supports storing the state of all memory, z80 and 8253 at the point in time when the key is pressed.

This state can be restored later by using the F11 function key.

The memory dump file is based on the .mzf format with extensions and omissions. As such, they are stored on microSD cards with the extension .mzd in the /memorydumps directory.

At the time of writing this format is still subject to change, so **should not** be used as a permanent storage mechanism for your Pico MZ programs.

Memory dump files from emulator version 3.2.0 and later cannot be used with earlier releases of the emulator and vice-versa.

Memory dump files produced by the Pico and Pico 2 are not cross-compatible, even for the same Pico MZ emulator.

Memory dump files produced by the MZ-80K, MZ-80A and MZ-700 emulations are not cross-compatible.

## Pico MZ memory dump file format as of November 2026 (v3.2.0).

### *Header*

The first 128 bytes of the file. The first byte stores the value 0x20, to identify that this is a Pico MZ format memory dump file. The next 16 bytes are used to store a file name (in the same way that a .mzf file does). These are:

| 0x4d | 0x92 | 0xb3 | 0xb7 | 0x9d | 0xbd | 0x20 | 0x9c | 0xa5 | 0xb3 | 0x9e | 0x20 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| M | e | m | o | r | y | <sp> | d | u | m | p | <sp> |

Followed by one of:

| 0x38 | 0x30 | 0x4b | 0x0d |
|---|---|---|---|
| 8 | 0 | K | <cr> |
| 0x38 | 0x30 | 0x41 | 0x0d |
| 8 | 0 | A | <cr> |
| 0x37 | 0x30 | 0x30 | 0x0d |
| 7 | 0 | 0 | <cr> |

depending on the model being emulated.

The remainder of the header block is undefined and unused.


Copyright © 2024-2026 Tim Holyoake.

### *Monitor ROM or (MZ-80A / MZ-700) 4K banked RAM*

The next 4096 bytes store either the contents of the monitor ROM, or on an MZ-80A or MZ-700, the 4K banked RAM area if the monitor ROM has been paged out.

### *Monitor workarea and user RAM*

The next 49,152 bytes. Populated with the contents of the monitor workarea and user RAM at the time the F12 key is pressed.

### *Video RAM*

The next 4096 bytes. Note that only the first 1024 bytes is valid on the MZ-80K and the first 2048 for the MZ-80A. Populated with the contents of the video RAM at the time the F12 key is pressed.

### *12K banked RAM (MZ-700 only)*

The next 12,228 bytes on dump files produced by the MZ-700 emulator. These are saved regardless of whether this portion of RAM was active.

### *Z80 state*

The next 56 bytes. Populated with the contents of the mzcpu global structure, used to maintain the state of the Z80 cpu.

### *8253 state*

The final 12 bytes. Populated with the contents of the mzpit global structure, used to maintain the state of the 8253 programmable interval timer (PIT).


Copyright © 2024-2026 Tim Holyoake.

# Release Highlights

## October 2024: Release 1.0.0

MZ-80K emulation for a Pico on a Pimoroni VGA demo base.

A winner of the RC2024/10 event.

## November 2024: Release 1.1.0

Introduction of Pico 2 support on a Pimoroni VGA demo base for the MZ-80K emulation.

## January 2025: Release 1.2.0

Introduction of RC2014 RP2040 VGA terminal card support for the MZ-80K emulation.

## January 2025: Release 1.2.3

Introduction of RC2014 Pi Pico VGA terminal card support for the MZ-80K emulation.

## January 2025: Release 1.2.4

Japanese CGROM implementation for the MZ-80K emulation.

## February 2025: Release 2.0.0

MZ-80A emulation introduced, selected by holding down button A at power on.

## November 2025: Release 3.0.0

MZ-700 emulation introduced as a separate .uf2 file from the same codebase as the MZ-80K and MZ-80A emulations.

Diagnostic mode using terminal emulators as a USB keyboard substitute was removed to simplify code maintenance and testing.

## September 2026: Release 3.1.0

Consolidated all emulators into a single .uf2 file per pico model / board type.

Japanese CGROM implementation for the MZ-700 emulation.

Multi-block BSD file support on MZ-80K/MZ-80A (type 0x03) and MZ-700 (type 0x04).

## November 2026: Release 3.2.0

Introduction of optional sub-directories to separate MZ-80K, MZ-80A and MZ-700 files.


Copyright © 2024-2026 Tim Holyoake.

# Acknowledgements

As well as directly including the third party libraries detailed in the architectural overview of the Pico MZ, some of the code was also inspired by other projects. These include:

The KM-Z80 MZ-80K emulator by Katsumi

A MZ-80 series emulator for Raspberry Pi by Nibbles Lab

VHDL implementations of Sharp MZ series computers by Philip Smart

Picoterm by RC2014

The RC2040 by Extreme Electronics

… and, of course, the people who run and take part in RetroChallenge. Much of the work completing the first version of this emulator was performed during the October 2024 event.

My own notes made during this time can be found at retrocomputing ephemera.

Sharp MZ-80K, MZ-80A, MZ-700 emulator

PICO MZ

https://z80.timholyoake.uk/

email: picomz@timholyoake.uk


Copyright © 2024-2026 Tim Holyoake.